Plan

Shared ML Core (Weeks 1–4): From Data Exploration to a First NLP Deep-Learning Model

Purpose & Outcomes

By the end of Week 4, you will be able to:

- Set up a reproducible ML environment and project scaffold.
- Load **OGBN-ArXiv (https://ogb.stanford.edu/docs/nodeprop/#ogbn-arxiv)** with the **official train/val/test split** and avoid data leakage.
- Perform focused EDA: target distribution, simple embedding visualization of node features, and basic graph structure histograms.
- Train and evaluate a **standard baseline** (multinomial Logistic Regression) with honest metrics.
- Build a **small deep-learning text classifier** and explain one meaningful improvement.
- Quantify and interpret the **generalization gap** (train vs validation/test) with side-by-side metrics and simple learning curves.

Who This Is For

Beginners who know basic Python. No prior ML experience required.

Tools (recommended)

- Python 3.10+, Conda/virtualenv
- PyTorch, scikit-learn, matplotlib/plotly
- Hugging Face Transformers (optional, for Week 3–4 text model)
- OGB + PyTorch Geometric for convenient data loading

Guardrails for Everyone

- Use the **official OGBN-ArXiv split** (time-based); do not create custom splits.
- Report **Accuracy** and **Macro-F1** on the **validation** set during model selection.
- Keep runs reproducible (fixed seeds, saved configs, minimal README).
- For model selection, compare **train vs validation**; keep the **test set** held out. If you check test, do it **once** after freezing a model and record it explicitly.

Schedule at a Glance (4 Weeks)

Weeks 1–2 — Setup, EDA, and an Honest Baseline

Focus: classic data-science flow; no deep learning yet.

Progress targets (Weeks 1–2)

- **EDA:**
  - Target distribution across the 40 classes
  - Simple 2-D visualization of the provided 128-d node features (e.g., PCA/UMAP)
  - Basic graph structure views: degree histograms, connectedness snapshot
  - 3–5 key takeaways
- **Baseline (validation):**
  - Multinomial **Logistic Regression** on the 128-d features
  - Metrics: **train vs validation** Accuracy + Macro-F1 (+ confusion matrix); include a simple **learning-curve**; optional **one-time test probe** after freezing the baseline
  - Small param scan (e.g., regularization strength)

By end of Week 2, you should have:

- Environment & dataset ready; official split verified
- EDA figures saved (target distribution, 2-D feature viz, degree histograms)
- Baseline Logistic Regression trained with a small param scan
- **Generalization gap** summarized (train vs validation) and a simple learning curve

---

Weeks 3–4 — Hyperparam Search → First DL (Simplest) → Training Knobs

Focus: experience the ML→DL transition with a small, practical text model.

Progress targets (Weeks 3–4)

- **Hyperparam search (start here):** finish a small, targeted sweep on the Week-2 baseline to lock a strong reference (e.g., C/max_iter for LR, or simple feature tweaks).
- **First DL (simplest):** train a minimal model
  - **Tiny MLP** on the 128-d features
- **Training knobs (one at a time):** add and compare **LR schedulers** (step/cosine/warmup), **optimizer** swap (AdamW vs SGD+momentum), and **normalization** where applicable (BatchNorm for MLP, LayerNorm if supported). Consider **dropout** and **weight decay**.
- **Eval parity:** same metrics as Weeks 1–2, reported **train vs validation**; optional one-time test probe only after the model is frozen.
- **Brief error notes:** 3–5 bullets tying failures to data patterns (e.g., rare classes, short texts, year slices).

By end of Week 4, you should have:

- A **completed hyperparam search** on the Week-2 baseline (locked reference)
- A **simplest DL model** trained
- Evidence of **at least one training knob** explored (scheduler/optimizer/normalization) with a short comparison
- Metrics reported **train vs validation** with a calibration check; optional single test probe after freezing

---

Week 5 (Mapping & Sanity check)

Focus: map the text to the node, and make sure the mapping is legit. (This is important!)

Progress targets

- Build a deterministic join from node_index → paper_id, then to your title/abstract table.
- Canonicalize keys: strip version suffixes like v\d+, trim, lowercase, normalize Unicode.
- Create a single cached table arxiv_text.parquet with at least:

  node_index, paper_id, year, text (title + abstract), label, split.

1. Please also map the label and read a few entries to see whether the label make sense.

Week 6-8

Focus: Explore different text encoding method with a linear probe as downstream and observe the difference in performance.

Progress targets:

1. **TF-IDF**
    - Use TF-IDF (w/wo n-grams (n=2,3,5)) to encode the abstract and use a logistic regression as the prediction head.
    - You can try to use explore some traditional NLP techniques like stopwords, lemming, etc. to do the preprocessing.
    - Please record the result and try to explain what makes the difference.
2. **Word2vec**
    - Use a pretrained word2vec model (e.g., word2vec-google-news-300) to encode the abstract and a logistic regression as the prediction head.
    - Record the result.
    - (optional) if you are interested you can try your own model on the training set and do the same experiment and see the difference.
3. **Modern encoding method**
    - Try modern embedding model (i.e., EmbeddingGemma, GPT4) to encode and do the same experiment.

By end of Week 8, you should have:

Finish all the experiment and present the performance under the 3 different settings.

Week 8-12

Focus: Try some more modern methods.

Progress targets:

1. **Frozen transformer**
    - Load **DistilBERT** (or BERT-base if VRAM allows), **freeze encoder**, train **linear head** only.
2. Light fine-tuning
    - Unfreeze the last few layers or full model if VRAM is ok and fine-tune the model.
    - Try LORA if the fine-tuning is not stable.
3. Record all the results and summarize what we have done this semester.

By end of Week 12, you should have:

Finish all the experiment and have a detailed report of all techniques we tried this term.